Plano de Testes: Automação API ServeRest (Challenge 3)

Plano de Testes - Challenge 03 API ServeRest

---

1. Índice

- Plano de Testes - Challenge 03 API ServeRest
- 1. Índice
- 2. Informações Gerais
- 3. Plano de Testes
  - 3.1 Escopo
    - Funcionalidade Testadas:
    - Fora do Escopo:
  - 3.2 Estratégia de Testes
- 4. Lista de Cenários Executados
  - 4.1 Módulo: Autenticação (CT-01 ao CT-03)
  - 4.2 Módulo: Cadastro de Usuários (CT-04 ao CT-05)
  - 4.4 Módulo: Atualização de Usuários (CT-10 ao CT-12)
  - 4.3 Módulo: Busca de Usuários (CT-06 ao CT-09)
- 5. Evidências de Execução
    - 5.1 Evidências de Sucesso (CT-01 ao CT-10)
      - CT-01: Login com sucesso
      - CT-04: Cadastro de usuário
      - CT-06: Busca por ID
    - 5.2 Evidência de Bug (CT-11)
    - 4.3 Evidência de Bug (CT-11)
  - 6 Lista de Bugs Encontrados
    - BUG-12: Atualização de usuário permite nome e password vazios
      - Passos para Reproduzir:
      - Resultado Esperado:
      - Resultado Obtido (BUG):
      - Evidência:
- 7. Checks Automatizados
    - 7.1 Criação da Feature Branch
  - 8. Como Executar os Testes
      - Pré-requisitos:
      - requirements.txt:
      - Execução:
      - Visualizar Resultados:
    - 8.1 Guia Rápido de Prefixos Semânticos
      - Fonte: iuricode/padroes-de-commits: Padrões de commits
  - Autor:

---

2. Informações Gerais

Item	Descrição
Objetivo	Validar funcionalidade criticas da API ServeRest através de testes automatizados com robot Framework, identificando bugs e garantindo a qualidade do sistema.

API Alvo	https://compassuol.serverest.dv
Ferramentas	Robot Framework, RequestsLibrary, Git
Total de Cenários	12 (CT-01 AO CT-12)
Cenários Executados	12 (100%)
Passou PASSED	10
Falhou FAILED/BUG	2
Bugs Identificados	2 (Bug-12)
Status Final	Reprovado (Bugs Críticos em Validação de Campos)
Link do Repositório	[arlendfs/qa-air-compass-uol at feature/challenge-automation-tests-api]

3. Plano de Testes

3.1 Escopo

Funcionalidade Testadas:

✅ Autenticação e Login (CT-01 ao CT-03)

✅ Cadastro de Usuários (CT-04 ao CT-05)

✅ Busca de Usuários por ID (CT-06)

✅ Listagem de Usuários (CT-07)

✅ Busca de Usuários por ID Válido/Inválido (CT-08 ao CT-09)

✅ Atualização de Usuários (CT-10 ao CT-12)

Fora do Escopo:

Testes de Produtos (próxima fase) Performance e Carga

Testes de Carrinhos (próxima fase) Performance e Carga

Testes de Performance e Carga

3.2 Estratégia de Testes

Tipo de Teste	Descrição	Cenários
Teste Positivo	Validar fluxos de sucesso	CT-01, CT-04, CT-06, CT-07, CT-08, CT-10

Teste Negativo	Validar tratamento de erros	CT-02, CT-05, CT-09, CT-11, CT-12
Validação de Contrato	Verificar estrutura de resposta	CT-03, CT-06

---

4. Lista de Cenários Executados

4.1 Módulo: Autenticação (CT-01 ao CT-03)

ID	Título	Tipo	Status	Descrição
CT-01	Login com credenciais válidas e extrair token	Positivo	PASSED	Validar autenticação e obtenção do Bearer Token
CT-02	Login com senha incorreta	Negativo	PASSED	Validar bloqueio de acesso com credenciais inválidas
CT-03	Validar contrato da resposta de login	Contrato	PASSED	Verificar estrutura e campos obrigatórios do JSON

4.2 Módulo: Cadastro de Usuários (CT-04 ao CT-05)

ID	Título	Tipo	Status	Descrição
CT-04	Cadastrar novo usuário com dados válidos	Positivo	PASSED	Criar usuário e validar geração de ID
CT-05	Tentar cadastrar usuário com e-mail duplicado	Negativo	PASSED	Validar bloqueio de e-mail já existente

---

4.4 Módulo: Atualização de Usuários (CT-10 ao CT-12)

ID	Título	Tipo	Status	Descrição
CT-10	Atualizar usuário com sucesso	Positivo	PASSED	Validar atualização de dados
CT-11	**[BUG]** Atualizar usuário com nome vazio	Negativo	FAILED	**BUG-12:** API permite nome vazio (Status 200)
CT-12	**[BUG]** Atualizar usuário com password vazio	Negativo	FAILED	**BUG-12:** API permite password vazio (Status 200)

4.3 Módulo: Busca de Usuários (CT-06 ao CT-09)

ID	Título	Tipo	Status	Descrição
CT-06	Buscar usuário por ID e validar campos obrigatórios	Contrato	✅ PASSED	Validar estrutura do usuário retornado
CT-07	Listar todos os usuários	Positivo	✅ PASSED	Verificar listagem completa
CT-08	Buscar usuário por ID válido	Positivo	✅ PASSED	Validar busca específica por ID
CT-09	Buscar usuário com ID inexistente	Negativo	✅ PASSED	Validar tratamento de erro para ID inválido

5. Evidências de Execução

5.1 Evidências de Sucesso (CT-01 ao CT-10)

CT-01: Login com sucesso

```
{
  "message": "Login realizado com sucesso",
  "authorization": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Status: ✅ 200 OK

CT-04: Cadastro de usuário

```
{
  "message": "Cadastro realizado com sucesso",
  "_id": "66f5a8b7c3d9e4f2a1b6c7d8"
}

```

Status: ✅ 201 Created

CT-06: Busca por ID

```
{
  "nome": "Arlen Bug Hunter",
  "email": "arlen.abc123@teste.com",
  "password": "123456",
  "administrador": "true",
  "_id": "66f5a8b7c3d9e4f2a1b6c7d8"
}
```

Status: ✅ 200 OK

5.2 Evidência de Bug (CT-11)

```
Tests.01 Login :: Testes de Login para a API ServeRest
==========================================================================
CT-01: Login Com Credenciais Válidas :: Verifica se o login é bem-... | PASS |
--------------------------------------------------------------------------
CT-02: Login Com Senha Inválida :: Verifica se o login falha com u... | PASS |
--------------------------------------------------------------------------
CT-03: Login Com Email Com Formato Inválido :: Verifica se o login... | PASS |
--------------------------------------------------------------------------
Tests.01 Login :: Testes de Login para a API ServeRest               | PASS |
3 tests, 3 passed, 0 failed
==========================================================================
Tests.02 Usuarios
==========================================================================
CT-04: Cadastrar Novo Usuário Administrador Com Sucesso :: Verific... | PASS |
--------------------------------------------------------------------------
CT-05: Tentar Cadastrar Usuário Com E-mail Já Existente :: Verific... | PASS |
--------------------------------------------------------------------------
CT-06: Buscar Usuário por ID E Validar Campos Obrigatórios :: Veri... | PASS |
--------------------------------------------------------------------------
Tests.02 Usuarios                                                    | PASS |
3 tests, 3 passed, 0 failed
==========================================================================
Tests.03 Produtos
==========================================================================
CT-07: Cadastrar Novo Produto Com Sucesso :: Verifica se é possíve... | PASS |
--------------------------------------------------------------------------
CT-08: Tentar Cadastrar Produto Com Nome Duplicado :: Verifica se ... | PASS |
CT-09: Listar Todos Produtos E Validar Produto Cadastrado :: Verif... | PASS |
--------------------------------------------------------------------------
Tests.03 Produtos                                                    | PASS |
3 tests, 3 passed, 0 failed
==========================================================================
Tests.04 Usuarios Update
==========================================================================
CT-10: Atualizar Usuário Com Sucesso :: Validar atualização de um ... | PASS |
--------------------------------------------------------------------------
CT-11: [BUG] Atualizar Usuário Com Nome Vazio :: Verificar se API ... | PASS |
--------------------------------------------------------------------------
CT-12: [BUG] Atualizar Usuário Com Password Vazio :: Verificar se ... | PASS |
--------------------------------------------------------------------------
Tests.04 Usuarios Update                                             | PASS |
3 tests, 3 passed, 0 failed
==========================================================================
Tests                                                                | PASS |
12 tests, 12 passed, 0 failed
==========================================================================
```

4.3 Evidência de Bug (CT-11)

Requisição:

```
PUT /usuarios/66f5a8b7c3d9e4f2a1b6c7d8 HTTP/1.1
Content-Type: application/json

{
  "nome": "",
  "email": "novo.email@teste.com",
  "password": "123456",
  "administrador": "true"
}
```

Resposta Obtida (BUG):

```
{
  "message": "Registro alterado com sucesso"
}
```

Status: 200 OK (Esperado: 400 Bad Request)

---

6 Lista de Bugs Encontrados

BUG-12: Atualização de usuário permite nome e password vazios

Campo	Detalhe
Severidade	Crítica
Impacto	Permite cadastrar usuários sem dados obrigatórios, comprometendo a integridade dos dados
Módulo	Usuários - PUT /usuarios/{id}

Passos para Reproduzir:

1. Criar um usuário válido via POST /usuarios
2. Obter o _id do usuário criado
3. Enviar requisição PUT para /usuarios/{id} com nome vazio
4. Observar o retorno da API

Resultado Esperado:

- Status Code: 400 Bad Request
- Mensagem: "O campo nome é obrigatório"

Resultado Obtido (BUG):

- Status Code: 200 OK
- Mensagem: "Registro alterado com sucesso"
- O nome do usuário é atualizado para vazio no banco de dados

Evidência:

```
// Requisição
{
  "nome": "",
  "email": "teste@email.com",
  "password": "123456",
  "administrador": "true"
}

// Resposta (BUG)
{
  "message": "Registro alterado com sucesso"
}
```

---

7. Checks Automatizados

7.1 Criação da Feature Branch

Check	O que valida	Como valida
CT-01	Autenticação	Verifica se o token é retornado e tem prefixo

		"Bearer"
CT-02	Bloqueio de acesso	Valida status 401 e mensagem de erro específica
CT-03	Contrato de Login	Valida campos `message` e `authorization` no JSON
CT-04	Cadastro de usuário	Verifica status 201 e presença do `_id`
CT-05	Validação de unicidade	Tenta cadastrar com e-mail existente e valida erro 400
CT-06	Contrato do usuário	Verifica campos obrigatórios: nome, e-mail, password, administrador, _id
CT-07	Listagem	Valida estrutura com `quantidade` e array `usuarios`
CT-08	Busca por ID	Verifica se dados retornados correspondem ao cadastro
CT-09	Tratamento de erro	Valida status 400 e mensagem para ID inexistente
CT-10	Atualização	Verifica status 200 e persistência dos dados alterados
CT-11	**BUG**	Detecta se API permite nome vazio (falha proposital)
CT-12	**BUG**	Detecta se API permite password vazio (falha proposital)

8. Como Executar os Testes

Pré-requisitos:

```
# Instalar Python 3.8+
# Instalar Robot Framework e dependências
pip install -r requirements.txt
```

requirements.txt:

```
robotframework
robotframework-requests
robotframework-jsonlibrary
```

Execução:

```
# Executar todos os testes
robot -d ./logs ./tests/

# Executar testes específicos
robot -d ./logs ./tests/01_login.robot
robot -d ./logs ./tests/02_usuarios.robot
robot -d ./logs ,tests/03_produtos.robot
robot -d ./logs ,tests/04_usuarios_update.robot
```

Visualizar Resultados:

```
# Abrir relatório no navegador
open ./logs/report.html
```

- Estrutura de Recursos (Keywords):

```
# Após criar seus arquivos .resource
git add resources/
git commit -m "feat: Implementação da api base keywords e gerenciamento
```

- Implementação dos Testes:

```
# Após criar os primeiros arquivos .robot
git add tests/
git commit -m "test: add teste funcional para registro de usuario e logi
```

- Finalização e Merge

```
# 1. Voltar para a main
git checkout main

# 2. Trazer as alterações (Meger)
git merge feature/challenge-automation-tests-api

# 3. Criar a Tag de Release (Exigência do Challenge 3)
git tag -a v1.0-challenge3 -m "Final delivery: Conjunto de Automação de

# 4. Enviar para o (Github/Gitlab)
git push origin main --tags
```

8.1 Guia Rápido de Prefixos Semânticos

Fonte: [iuricode/padroes-de-commits: Padrões de commits]

Prefixo	Quando usar
feat	Commits do tipo feat indicam que seu trecho de código está incluindo um **novo recurso** (se relaciona com

	o MINOR do versionamento semântico).
fix	Commits do tipo fix indicam que seu trecho de código commitado está **solucionando um problema** (bug fix), (se relaciona com o PATCH do versionamento semântico).
docs	Commits do tipo docs indicam que houveram **mudanças na documentação**, como por exemplo no Readme do seu repositório. (Não inclui alterações em código).
test	Commits do tipo test são utilizados quando são realizadas **alterações em testes**, seja criando, alterando ou excluindo testes unitários. (Não inclui alterações em código).
build	Commits do tipo build são utilizados quando são realizadas modificações em **arquivos de build e dependências**.
perf	Commits do tipo perf servem para identificar quaisquer alterações de código que estejam relacionadas a **performance**.
style	Commits do tipo style indicam que houveram alterações referentes a **formatações de código**, semicolons, trailing spaces, lint... (Não inclui alterações em código).
refactor	Commits do tipo refactor referem-se a mudanças devido a **refatorações que não alterem sua funcionalidade**, como por exemplo, uma alteração no formato como é processada determinada parte da tela, mas que manteve a mesma funcionalidade, ou melhorias de performance devido a um code review.

chore	Commits do tipo chore indicam **atualizações de tarefas** de build, configurações de administrador, pacotes... como por exemplo adicionar um pacote no gitignore. (Não inclui alterações em código).
ci	Commits do tipo ci indicam mudanças relacionadas a **integração contínua** (*continuous integration*).
raw	Commits do tipo raw indicam mudanças relacionadas a arquivos de configurações, dados, features, parâmetros.
cleanup	Commits do tipo cleanup são utilizados para remover código comentado, trechos desnecessários ou qualquer outra forma de limpeza do código-fonte, visando aprimorar sua legibilidade e manutenibilidade.
remove	Commits do tipo remove indicam a exclusão de arquivos, diretórios ou funcionalidades obsoletas ou não utilizadas, reduzindo o tamanho e a complexidade do projeto e mantendo-o mais organizado

---

Autor:

Arlen Freitas - Squad 3